De los problemas NP-hard a las aplicaciones actuales

19 problemas fundacionales · 18 reducciones

Descripción, entradas, salidas, ejemplos y conexiones con problemas de hoy.

Organizar exámenes, elegir proyectos, distribuir trabajos entre servidores y diseñar rutas son tareas diferentes que pueden compartir una misma estructura matemática.

Reconocer esa estructura permite transformar una aplicación en un problema conocido y aprovechar sus algoritmos, sus herramientas y sus límites.

Objetivo de aprendizaje. Identificar decisiones, restricciones y objetivos; construir un modelo; seleccionar un problema fundacional y traducir la solución de vuelta a la aplicación.

Público. Estudiantes que comienzan el estudio de algoritmos y complejidad. Se supone familiaridad básica con grafos, conjuntos y lógica booleana.

Basada en el esquema «Eighteen Reductions», atribuido a Tim Roughgarden en la imagen proporcionada. Las aplicaciones y ejemplos fueron elaborados con fines didácticos.

Cómo leer esta guía

El mapa tiene 19 problemas y 18 flechas

El título de la imagen cuenta reducciones, no problemas. Esta guía dedica una ficha a cada uno de los 19 problemas y después explica las 18 conexiones. El propósito es motivar el modelado; las descripciones de las reducciones no sustituyen sus demostraciones formales.

Decisión, búsqueda y optimización

Versión	Pregunta de ejemplo
Decisión	¿Existe una ruta de costo como máximo 100?
Búsqueda	Encuentra una ruta que cumpla ese límite.
Optimización	Encuentra la ruta de costo mínimo.

Qué significa NP-hard

Un problema es NP-hard si todo problema de NP puede reducirse a él en tiempo polinómico. Si pudiéramos resolverlo exactamente para todas las entradas en tiempo polinómico, podríamos resolver de esa forma todos los problemas de NP. No se conoce tal algoritmo y tampoco se ha demostrado que sea imposible: esta es la cuestión P versus NP.

Un problema de decisión es NP-completo si es NP-hard y además pertenece a NP: sus respuestas afirmativas admiten certificados verificables en tiempo polinómico. Las versiones de optimización se describen normalmente como NP-hard.

La dirección de una reducción

A $\leq_p$ B significa que una instancia de A se transforma en una instancia de B en tiempo polinómico, conservando la respuesta de decisión. Por eso, un algoritmo para B permite resolver A. Para recuperar una solución concreta, también debemos saber cómo traducirla.

Aplicación A	Transformación	Algoritmo B	Interpretación
Horarios de exámenes	Exámenes → vértices; conflictos → aristas	Coloreo de grafos	Colores → bloques horarios

Una reducción de decisión no transfiere automáticamente garantías de aproximación. Tampoco garantiza que la instancia transformada sea fácil para un solucionador práctico.

3-SAT

Satisfacer decisiones lógicas

Descripción. Busca una asignación de verdadero o falso que cumpla una fórmula formada por cláusulas de tres literales. Un literal es una variable o su negación. Cada cláusula exige que al menos uno de sus literales sea verdadero; deben cumplirse todas las cláusulas.

Entradas. Variables booleanas y una conjunción de cláusulas, cada una con tres literales.

Salidas. Una asignación satisfactoria, si existe, o la respuesta «no satisfacible».

Un ejemplo pequeño

(x ∨ y ∨ z) ∧ (¬x ∨ y ∨ ¬z). La asignación x = verdadero, y = verdadero, z = falso satisface ambas cláusulas.

Aplicaciones y problemas similares

Configuración de productos, compatibilidad de componentes, selección de características de software y verificación de circuitos. Una regla «si A, entonces B» se escribe como ¬A ∨ B; un sistema de reglas puede codificarse en SAT y transformarse en 3-SAT.

Cómo aprovechar un algoritmo

Convertir las reglas de una aplicación en cláusulas permite utilizar un solucionador lógico para buscar una configuración válida. Las variables auxiliares y sus valores deben interpretarse correctamente al recuperar la solución.

Para conversar en clase

¿Cómo expresarías «al menos una de estas tres opciones»? ¿Y «A y B no pueden elegirse juntas»?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

SAT

Un lenguaje común para restricciones

Descripción. Busca una asignación que haga verdadera una fórmula booleana. A diferencia de 3-SAT, no limita las cláusulas a tres literales. Muchos solucionadores reciben la fórmula en forma normal conjuntiva.

Entradas. Una fórmula booleana sobre un conjunto finito de variables.

Salidas. Una asignación satisfactoria o una indicación de que la fórmula es insatisfacible.

Un ejemplo pequeño

(a ∨ b) ∧ (¬a ∨ c) ∧ ¬c se satisface con a = falso, b = verdadero, c = falso.

Aplicaciones y problemas similares

Resolver dependencias de software, encontrar entradas que produzcan fallas en circuitos y verificar propiedades de sistemas dentro de un horizonte de ejecución acotado. En verificación se puede preguntar si existe una ejecución que viole una propiedad.

Cómo aprovechar un algoritmo

Un mismo solucionador SAT puede atender dominios diferentes cuando sus reglas se traducen a lógica. Una respuesta insatisfacible se refiere al modelo construido y, en verificación acotada, al alcance analizado.

Para conversar en clase

Si un conjunto de reglas es incompatible, ¿qué explicación necesitaría una persona para corregirlo?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

FICHA 03 / 19

MIP · programación entera mixta

Decisiones discretas con restricciones lineales

Descripción. Formula decisiones mediante variables, restricciones lineales y, en optimización, una función objetivo lineal. Algunas variables son enteras y otras pueden ser continuas. Las binarias representan elecciones de sí o no.

Entradas. Variables con sus dominios, restricciones lineales y una función objetivo opcional.

Salidas. Una solución óptima y su valor si existe un óptimo finito; también puede determinarse inviabilidad o falta de acotación del objetivo.

Un ejemplo pequeño

Maximizar $6x + 5y$, sujeto a $4x + 3y \leq 7$ y $x,y \in \{0,1\}$. La solución $x = y = 1$ obtiene valor 11.

Aplicaciones y problemas similares

Asignación de turnos, planificación de producción, apertura de centros logísticos, inversiones y recursos de cómputo. Por ejemplo, una variable binaria puede indicar si se abre un centro y otra continua cuánto se envía desde él.

Cómo aprovechar un algoritmo

El modelado permite utilizar un solucionador general. Si se limita su tiempo, puede devolver una solución factible y cotas sin haber demostrado optimalidad. MIP contiene como casos particulares muchos modelos combinatorios.

Para conversar en clase

En un problema de transporte, ¿qué cantidades deberían ser enteras y cuáles podrían ser continuas?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Coloreo de grafos

Asignar recursos evitando conflictos

Descripción. Asigna colores a los vértices de un grafo no dirigido de modo que los extremos de cada arista tengan colores diferentes. En optimización se minimiza el número de colores.

Entradas. Un grafo; en decisión, un límite k de colores.

Salidas. Una coloración válida mínima, o la respuesta sobre si existe una con como máximo k colores.

Un ejemplo pequeño

Tres exámenes que comparten estudiantes entre cada pareja forman un triángulo. Se requieren tres colores, interpretados como tres horarios.

Aplicaciones y problemas similares

Horarios de exámenes, asignación simplificada de frecuencias y asignación de registros en compiladores. En horarios, cada examen es un vértice y cada conflicto entre dos exámenes es una arista.

Cómo aprovechar un algoritmo

Un algoritmo de coloreo asigna recursos reutilizables a elementos que tienen incompatibilidades por pares. Capacidades de salas, disponibilidad y preferencias requieren restricciones adicionales. Dos colores admiten un algoritmo polinómico; decidir tres colores ya es NP-completo en grafos generales.

Para conversar en clase

¿Cómo incorporarías que un examen solo puede realizarse durante la mañana?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Conjunto independiente

Elegir elementos que no entren en conflicto

Descripción. Busca vértices sin aristas entre ninguno de sus pares. En optimización interesa el conjunto de mayor tamaño. Cada arista representa una incompatibilidad.

Entradas. Un grafo no dirigido; en decisión, un tamaño mínimo k.

Salidas. Un conjunto independiente máximo, o la respuesta sobre si existe uno de tamaño al menos k.

Un ejemplo pequeño

En el camino A—B—C—D, el conjunto {A,C} es independiente y tiene tamaño máximo 2.

Aplicaciones y problemas similares

Selección de actividades compatibles, transmisiones simultáneas sin interferencia y ubicaciones sujetas a incompatibilidades por pares. Si dos actividades no pueden coexistir, se conectan sus vértices.

Cómo aprovechar un algoritmo

La solución del grafo indica qué candidatos elegir juntos. Si sus beneficios son distintos, aparece la variante de peso máximo. Algunos casos estructurados, como grafos de intervalos para actividades, admiten algoritmos eficientes.

Para conversar en clase

¿Cambiaría la solución si una actividad valiera diez veces más que las demás?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Clique

Buscar compatibilidad entre todos los pares

Descripción. Una clique es un conjunto de vértices en el que cada pareja está conectada. Se busca una de tamaño máximo o de tamaño al menos k.

Entradas. Un grafo no dirigido; en decisión, un tamaño requerido k.

Salidas. Una clique máxima o la respuesta sobre la existencia de una clique de tamaño al menos k.

Un ejemplo pequeño

Las aristas AB, AC y BC hacen que {A,B,C} sea una clique de tamaño 3.

Aplicaciones y problemas similares

Grupos de compatibilidad mutua y correspondencias entre imágenes. En este último modelo, cada vértice puede representar una posible correspondencia y cada arista indica que dos correspondencias son compatibles.

Cómo aprovechar un algoritmo

Un algoritmo de clique selecciona hipótesis mutuamente compatibles. El modelo exige compatibilidad entre todos los pares; una comunidad social real no necesariamente cumple esa condición y podría requerir otra noción de agrupamiento.

Para conversar en clase

¿En qué aplicación sería demasiado restrictivo exigir que todos estén conectados con todos?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Cobertura de vértices

Atender todas las conexiones

Descripción. Selecciona vértices de modo que toda arista tenga al menos un extremo elegido. Se busca minimizar la cantidad de vértices.

Entradas. Un grafo no dirigido; en decisión, un presupuesto k.

Salidas. Una cobertura mínima, o la respuesta sobre si existe una de tamaño como máximo k.

Un ejemplo pequeño

En el camino A—B—C, seleccionar {B} cubre las dos aristas.

Aplicaciones y problemas similares

Ubicar puntos de inspección cuando revisar un extremo permite atender una conexión; seleccionar módulos para revisar cuando cada conflicto involucra exactamente dos módulos; atender interacciones eligiendo al menos uno de sus participantes.

Cómo aprovechar un algoritmo

Si cada obligación involucra dos candidatos y basta escoger uno, el modelo es cobertura de vértices. Cubrir todos los vértices mediante vecinos es otro problema, conjunto dominante. En grafos bipartitos, la cobertura mínima se obtiene en tiempo polinómico.

Para conversar en clase

¿Cómo representarías enlaces que quedan supervisados cuando se inspecciona cualquiera de sus dos extremos?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Set Cover · cobertura de conjuntos

Cubrir todas las necesidades

Descripción. Dado un universo de elementos y varios subconjuntos candidatos, selecciona la menor cantidad de conjuntos cuya unión cubra todo el universo.

Entradas. Un universo U y una familia de subconjuntos. Puede añadirse un costo por conjunto o un límite de selección.

Salidas. Una selección mínima que cubra U, o la indicación de que no es posible cubrirlo.

Un ejemplo pequeño

U = {1,2,3,4}; S_1 = {1,2}, S_2 = {2,3}, S_3 = {3,4}. Elegir S_1 y S_3 cubre todo con dos conjuntos.

Aplicaciones y problemas similares

Elegir pruebas de software que verifiquen todas las funciones requeridas, sensores que observen todas las zonas, cursos que cubran competencias o proveedores que suministren todos los componentes.

Cómo aprovechar un algoritmo

Cada acción se representa por las necesidades que satisface. Un algoritmo devuelve qué acciones seleccionar. Con costos diferentes se usa la variante ponderada. La calidad del resultado depende de cómo se definió la cobertura.

Para conversar en clase

Si una prueba cubre muchas funciones pero tarda una hora, ¿cómo debería cambiar el objetivo?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Maximum Coverage · cobertura máxima

Aprovechar un presupuesto limitado

Descripción. Selecciona como máximo k conjuntos para maximizar el número de elementos cubiertos. A diferencia de Set Cover, no exige cubrir todo: el presupuesto de selección está fijado.

Entradas. Un universo, una familia de subconjuntos y un presupuesto k. Los elementos pueden tener pesos.

Salidas. Una selección de hasta k conjuntos con cobertura máxima.

Un ejemplo pequeño

U = {1,2,3,4,5}; A = {1,2,3}, B = {3,4}, C = {4,5}. Con k = 2, A y C cubren los cinco elementos.

Aplicaciones y problemas similares

Ubicaciones de servicios que atiendan a la mayor población posible; pruebas de software con un límite de ejecución; selección de sensores; contenidos que cubran la mayor variedad de temas.

Cómo aprovechar un algoritmo

El algoritmo evalúa el aporte conjunto, descontando superposiciones. En el modelo habitual con presupuesto de cardinalidad, el método voraz que elige el mayor aporte marginal logra al menos una fracción $1-1/e$ del óptimo.

Para conversar en clase

¿Por qué elegir los dos conjuntos más grandes por separado podría ser peor que elegir uno grande y otro pequeño?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Maximización de influencia

Elegir dónde iniciar una difusión

Descripción. Selecciona nodos iniciales o semillas para maximizar la propagación en una red. La difusión puede ser determinista o probabilística; es necesario especificar sus reglas.

Entradas. Una red generalmente dirigida, un presupuesto k y un modelo de difusión con sus parámetros.

Salidas. Un conjunto de semillas que maximice la cantidad de nodos activados, o su esperanza en un modelo probabilístico.

Un ejemplo pequeño

A activa con certeza a B y C; D activa a E. Con una semilla, elegir A activa tres nodos contando a A, mientras D activa dos.

Aplicaciones y problemas similares

Difusión de información pública, elección de promotores y distribución inicial de contenidos. Dos semillas con muchos contactos pueden aportar poco si llegan a las mismas personas.

Cómo aprovechar un algoritmo

Un algoritmo compara el alcance conjunto de las semillas. Su utilidad depende de la fidelidad del modelo. En modelos probabilísticos comunes, calcular exactamente la influencia esperada también puede ser difícil; se utilizan simulaciones o estimaciones.

Para conversar en clase

¿Por qué el nodo con más contactos no siempre es la mejor semilla?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Subset Sum · suma de subconjuntos

Encontrar una coincidencia exacta

Descripción. Dada una lista de enteros positivos, pregunta si algunos suman exactamente un objetivo. Cada posición de la lista puede seleccionarse como máximo una vez.

Entradas. Una lista $a_1,\ldots,a_n$ y un objetivo T.

Salidas. Un subconjunto de posiciones cuya suma sea T, o la respuesta de que no existe.

Un ejemplo pequeño

Lista {3,5,6,9}, objetivo 14. Una solución es seleccionar 5 y 9.

Aplicaciones y problemas similares

Conciliación de pagos para identificar qué facturas suman un depósito, selección de lotes para completar una cantidad y combinación de recursos que deben alcanzar un total exacto. En dinero conviene usar unidades enteras, como centavos.

Cómo aprovechar un algoritmo

Una coincidencia exacta entre cantidades se transforma directamente en Subset Sum. Existe programación dinámica seudopolinómica respecto de T: puede ser práctica para objetivos moderados, pero no es polinómica respecto de los bits necesarios para escribir T.

Para conversar en clase

¿Por qué un objetivo enorme puede dificultar la solución aunque la cantidad de números sea la misma?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Mochila 0/1 · Knapsack

Maximizar beneficio con capacidad limitada

Descripción. Cada objeto tiene peso y valor. Se eligen objetos completos, cada uno como máximo una vez, sin superar una capacidad y maximizando el valor total.

Entradas. Pesos, valores y capacidad W.

Salidas. Un subconjunto de objetos con peso total como máximo W y valor máximo.

Un ejemplo pequeño

A tiene peso 2 y valor 6; B, peso 3 y valor 10; C, peso 4 y valor 12. Con capacidad 5, A y B logran el óptimo: valor 16 y peso 5.

Aplicaciones y problemas similares

Proyectos con presupuesto limitado, archivos para una caché, funcionalidades según esfuerzo disponible y equipos para una expedición. El recurso limitado es el peso y el beneficio es el valor.

Cómo aprovechar un algoritmo

El algoritmo devuelve qué opciones tomar. El modelo básico supone consumos y beneficios aditivos. Dependencias, sinergias y varios recursos exigen variantes. La mochila fraccionaria, que permite dividir objetos, es un problema diferente con un algoritmo voraz eficiente.

Para conversar en clase

¿Qué restricción agregarías si un proyecto solo pudiera elegirse junto con otro?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Minimización del makespan

Terminar toda la carga lo antes posible

Descripción. Distribuye trabajos entre máquinas para minimizar el tiempo de finalización del último trabajo. Aquí las máquinas son idénticas, los trabajos están disponibles al inicio y no pueden dividirse ni interrumpirse.

Entradas. La duración de cada trabajo y el número m de máquinas.

Salidas. Una asignación que minimice la mayor carga de una máquina, llamada makespan.

Un ejemplo pequeño

Trabajos {3,3,2,2} y dos máquinas. Poner un trabajo de duración 3 y uno de duración 2 en cada máquina logra makespan 5. Es óptimo: el trabajo total es 10 y hay dos máquinas.

Aplicaciones y problemas similares

Tareas independientes en servidores, lotes de producción, renderizado y procesamiento de datos. Cuando las máquinas son equivalentes, equilibrar la carga reduce la finalización global.

Cómo aprovechar un algoritmo

Un algoritmo de scheduling transforma una lista de trabajos en una asignación. Precedencias, transferencias, velocidades distintas o llegadas de nuevos trabajos requieren modelos adicionales. La mayor duración individual y la carga total dividida entre m son cotas inferiores útiles.

Para conversar en clase

¿Qué información faltaría si algunas tareas solo pudieran empezar al terminar otras?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Camino hamiltoniano dirigido

Visitar cada vértice una vez respetando sentidos

Descripción. Busca un camino que visite todos los vértices exactamente una vez y respete la orientación de los arcos. No exige regresar al inicio.

Entradas. Un grafo dirigido.

Salidas. Una secuencia de todos los vértices conectados por arcos en el sentido correcto, o la indicación de que no existe.

Un ejemplo pequeño

Con arcos A→B, B→C y A→C, la secuencia A→B→C es un camino hamiltoniano.

Aplicaciones y problemas similares

Secuencias de operaciones con transiciones permitidas, recorridos con conexiones de un solo sentido y ordenamientos que deben utilizar cada elemento una vez.

Cómo aprovechar un algoritmo

Se crea un vértice por entidad y un arco por transición admisible. Un camino entrega una secuencia válida. «Puedo pasar de A a B» no significa lo mismo que «A debe ocurrir antes que B»: las precedencias no se modelan automáticamente de esta forma. En grafos dirigidos acíclicos, el problema admite un tratamiento polinómico mediante caminos más largos.

Para conversar en clase

¿Por qué un recorrido que visita todos los lugares pero repite uno no cumple la definición?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Camino hamiltoniano no dirigido

Una visita por ubicación, en cualquier sentido

Descripción. Busca un camino que visite todos los vértices exactamente una vez. Las aristas pueden recorrerse en ambos sentidos y no se exige cerrar un circuito.

Entradas. Un grafo no dirigido.

Salidas. Un camino hamiltoniano, si existe; en caso contrario, una respuesta negativa.

Un ejemplo pequeño

Con aristas AB, BC y CD, el camino A—B—C—D visita los cuatro vértices exactamente una vez.

Aplicaciones y problemas similares

Recorridos de inspección, ordenamiento de muestras con compatibilidad entre elementos consecutivos y rompecabezas de recorrido. Las conexiones representan movimientos o transiciones posibles.

Cómo aprovechar un algoritmo

El algoritmo ordena todos los elementos cuando las conexiones son simétricas y no se permiten repeticiones. Si una aplicación permite repetir ubicaciones o debe recorrer todas las aristas, necesita otra formulación. Visitar vértices y recorrer aristas son requisitos diferentes.

Para conversar en clase

¿Qué diferencia hay entre un camino hamiltoniano y un recorrido que usa todas las aristas una sola vez?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Camino más corto sin ciclos

Optimizar una ruta simple con recompensas

Descripción. Busca un camino de costo mínimo entre s y t sin repetir vértices. Esta condición define un camino simple. Para la NP-dificultad del esquema es esencial permitir costos negativos en grafos generales.

Entradas. Un grafo dirigido con costos, un origen s y un destino t; los costos pueden ser negativos.

Salidas. Un camino simple de costo mínimo de s a t, o la indicación de que no existe conexión.

Un ejemplo pequeño

Arcos s→a de costo 2, a→t de costo −5 y s→t de costo 0. El camino s→a→t cuesta −3 y es mejor que el directo.

Aplicaciones y problemas similares

Secuencias que combinan gastos y recompensas sin reutilizar estados; recorridos donde una visita otorga un beneficio; planificación que prohíbe repetir acciones.

Cómo aprovechar un algoritmo

Un algoritmo resolvería rutas con recompensas negativas y prohibición de repeticiones. Con costos no negativos, el camino más corto ordinario es polinómico. Si no hay ciclos negativos relevantes, también existen algoritmos polinómicos con aristas negativas. No debe confundirse esa situación con el caso general de esta ficha.

Para conversar en clase

Si cada arco cuesta −1, ¿buscar el menor costo favorece usar más o menos arcos?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

TSP · problema del viajante

Visitar todo y regresar al inicio

Descripción. Busca un circuito de costo mínimo que visite cada ciudad exactamente una vez y vuelva al punto de partida. Consideramos la versión simétrica: el costo es igual en ambos sentidos.

Entradas. Un conjunto de ciudades y sus costos por pares. En decisión se agrega un umbral B.

Salidas. Un tour de costo mínimo; en decisión, se responde si existe uno de costo como máximo B.

Un ejemplo pequeño

Cuatro ciudades forman un cuadrado. Cada lado cuesta 1 y cada diagonal 2. Recorrer el contorno logra costo 4, que es óptimo.

Aplicaciones y problemas similares

Inspecciones, orden de perforación de una placa, movimiento de herramientas y secuenciación con costos de preparación entre trabajos. Cada ciudad puede representar una operación, no necesariamente una ubicación física.

Cómo aprovechar un algoritmo

El algoritmo entrega un orden cíclico para todas las visitas. Reparto con varios vehículos, capacidades y ventanas horarias requiere variantes. La versión métrica cumple desigualdad triangular; sus garantías de aproximación no deben trasladarse sin más a costos arbitrarios.

Para conversar en clase

¿Qué cambia si se permite terminar en una ciudad distinta del origen?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Min-Cost k-Path

Elegir una cadena de longitud exacta

Descripción. Busca un camino simple de exactamente k aristas con costo mínimo. En esta guía k forma parte de la entrada y los extremos son libres. Otros textos pueden contar vértices o fijar extremos.

Entradas. Un grafo con costos y un entero k.

Salidas. Un camino simple de k aristas y costo mínimo, o una respuesta de inexistencia.

Un ejemplo pequeño

En A—B—C—D, los costos consecutivos son 2, 1 y 4. Para k = 2, el camino A—B—C cuesta 3 y es el mejor.

Aplicaciones y problemas similares

Itinerarios con un número exacto de etapas, cadenas de transiciones de longitud fijada y secuencias que deben visitar cierta cantidad de ubicaciones diferentes.

Cómo aprovechar un algoritmo

El algoritmo controla simultáneamente longitud, costo y ausencia de repeticiones. Si se permiten repeticiones, un recorrido de exactamente k aristas se puede calcular mediante programación dinámica en tiempo polinómico en k y el tamaño del grafo. Si k es una constante fija, también puede enumerarse un camino simple en tiempo polinómico respecto del tamaño del grafo.

Para conversar en clase

¿Por qué debe especificarse si k cuenta vértices o aristas?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

Traveling Salesman Path

Visitar todo sin cerrar el circuito

Descripción. Busca un camino de costo mínimo que visite todas las ciudades exactamente una vez, sin exigir el regreso al inicio. Es la versión abierta del TSP.

Entradas. Ciudades y costos entre pares. Según la variante, los extremos pueden ser libres o estar fijados.

Salidas. Un camino hamiltoniano de costo mínimo que cumpla las condiciones de los extremos.

Un ejemplo pequeño

A, B y C están en una línea: AB = 1, BC = 1 y AC = 2. El camino A—B—C tiene costo 2 y es óptimo con extremos libres.

Aplicaciones y problemas similares

Inspecciones que terminan en otra ubicación, secuencias de fabricación sin retorno al estado inicial y planes de visita que deben cubrir todos los destinos sin cerrar el recorrido.

Cómo aprovechar un algoritmo

El algoritmo devuelve un orden abierto para las visitas obligatorias. La variante de extremos debe elegirse según la aplicación. Tampoco basta eliminar la arista más cara de un tour óptimo para garantizar un camino abierto óptimo: el mejor orden puede cambiar.

Para conversar en clase

¿Cómo cambia el modelo al fijar el origen, el destino o ambos?

Piensa en una aplicación propia: ¿qué representan las variables, los vértices, los conjuntos o los costos?

EL MAPA DE LAS REDUCCIONES

Conexiones 1 a 6

La flecha A → B indica que un algoritmo para B puede utilizarse para resolver A después de transformar la entrada. Estas son ideas de construcción, no pruebas completas.

01. 3-SAT → SAT

Toda fórmula de 3-SAT ya es una instancia de SAT. No hace falta modificarla.

02. SAT → MIP

Para una fórmula en forma conjuntiva, se usan variables binarias. Cada cláusula se expresa mediante una desigualdad: $x \vee \neg y \vee z$ equivale a $x + (1-y) + z \geq 1$. Una fórmula general puede convertirse primero a forma conjuntiva con variables auxiliares y tamaño polinómico.

03. 3-SAT → Coloreo de grafos

Se construyen dispositivos de variables y cláusulas que permiten una coloración con tres colores exactamente cuando la fórmula es satisfactoria. Los colores codifican elecciones lógicas y su consistencia.

04. 3-SAT → Conjunto independiente

Cada cláusula produce un triángulo con sus tres apariciones de literales. Se conectan además apariciones de literales contradictorios. Si hay m cláusulas, un conjunto independiente de tamaño m elige un literal compatible por cláusula.

05. Conjunto independiente → Clique

Se toma el complemento del grafo: los pares distintos que no eran adyacentes pasan a serlo y viceversa. Un conjunto independiente de tamaño k pasa a ser una clique de tamaño k.

06. Conjunto independiente → Cobertura de vértices

S es independiente si y solo si $V \setminus S$ cubre todas las aristas. Con n vértices, preguntar por un independiente de tamaño al menos k equivale a preguntar por una cobertura de tamaño como máximo $n-k$.

Conexiones 7 a 12

La flecha A → B indica que un algoritmo para B puede utilizarse para resolver A después de transformar la entrada. Estas son ideas de construcción, no pruebas completas.

07. Conjunto independiente → Subset Sum

Se codifican las selecciones y sus restricciones mediante posiciones de dígitos y números auxiliares, evitando acarreos no previstos. Alcanzar un objetivo equivale a una selección válida. El tamaño relevante es la cantidad de bits, que debe ser polinómica.

08. Subset Sum → Mochila

Cada número se convierte en un objeto cuyo peso y valor son ese número. Se fija capacidad T y se pregunta si puede obtenerse valor al menos T. La capacidad obliga a que la suma sea exactamente T.

09. Subset Sum → Makespan

Se transforma primero en partición: si los números suman S y $0 \le T \le S$, se agregan $2S-T$ y $S+T$. El nuevo total es $4S$. Una partición en mitades de $2S$ separa los dos números añadidos y obliga a completar T con originales. Los números son trabajos para dos máquinas; makespan $\le 2S$ equivale a la partición. Los casos triviales se manejan aparte.

10. Cobertura de vértices → Set Cover

El universo son las aristas. Cada vértice produce el conjunto de sus aristas incidentes. Elegir hasta k vértices que cubran las aristas equivale a elegir hasta k conjuntos que cubran el universo.

11. Set Cover → Maximum Coverage

Se conserva el presupuesto k y se pregunta si la máxima cobertura alcanza todos los elementos del universo. Si lo hace, existe una cobertura de tamaño como máximo k.

12. Maximum Coverage → Influencia

Una red de dos capas tiene nodos de conjuntos y de elementos, con arcos de activación determinista desde cada conjunto a sus elementos. Suponiendo cada elemento cubrible y k no mayor que el número de conjuntos, una semilla de elemento puede reemplazarse por un conjunto que lo contenga, sin empeorar el alcance; se completa hasta k semillas de conjuntos. El alcance óptimo es k más la máxima cobertura.

Conexiones 13 a 18

La flecha A → B indica que un algoritmo para B puede utilizarse para resolver A después de transformar la entrada. Estas son ideas de construcción, no pruebas completas.

13. 3-SAT → Camino hamiltoniano dirigido

Se construyen dispositivos que representan valores de verdad y verifican las cláusulas. Un camino que visita todos los vértices existe exactamente cuando las elecciones satisfacen la fórmula. La corrección requiere demostrar las dos implicaciones.

14. Camino hamiltoniano dirigido → No dirigido

Cada vértice se sustituye por una cadena de entrada, centro y salida; los arcos originales conectan salidas con entradas. Para controlar los extremos, se pueden añadir antes un origen con arcos a todos los vértices y un destino accesible desde todos, y después dos hojas que fuercen esos extremos. El recorrido simula los sentidos originales.

15. Camino hamiltoniano dirigido → Camino simple mínimo

Se añaden s y t auxiliares, con arcos de costo 0 desde s a cada vértice original y de cada original a t. Los arcos originales cuestan -1. Con n vértices originales, un camino simple s–t de costo $-(n-1)$ existe si y solo si hay un camino hamiltoniano original.

16. Camino hamiltoniano no dirigido → TSP

Se añade un vértice universal y se completa el grafo: aristas originales y conexiones al nuevo vértice cuestan 1; las demás cuestan 2. Con n vértices originales, un tour de costo $n+1$ existe si y solo si el grafo original tiene camino hamiltoniano.

17. TSP → Min-Cost k-Path

Se escoge una ciudad r y se sustituye por dos copias a y b, sin arista entre ellas; ambas heredan sus costos hacia las otras ciudades. Se conecta una hoja de costo 0 a cada copia. Para n ciudades hay $n+3$ vértices. Un camino simple con $k = n+2$ aristas visita todos, tiene extremos forzados en las hojas y reproduce un tour del mismo costo.

18. TSP → Traveling Salesman Path

Se utiliza la misma apertura en dos copias con dos hojas de extremos forzados. Para obtener una instancia completa, se asigna a las aristas prohibidas un costo M mayor que el umbral B de decisión, suponiendo costos originales no negativos. Así, un camino de costo $\leq B$ evita esas aristas y corresponde a un tour original. La construcción conserva tamaño polinómico.

Tres aplicaciones, paso a paso

A. Horarios de exámenes → Coloreo

Crear un vértice por examen y una arista entre exámenes que comparten estudiantes. Interpretar cada color como un bloque horario. Ejecutar un algoritmo de coloreo y convertir los colores en horarios. Una coloración válida evita estos conflictos; salas, capacidades y disponibilidad deben modelarse adicionalmente.

B. Pruebas de software → Cobertura

Las funciones requeridas forman el universo; cada prueba cubre las funciones que verifica. Si deben cubrirse todas usando pocas pruebas, usar Set Cover. Si solo hay presupuesto para cinco pruebas, usar Maximum Coverage. La solución es la lista de pruebas a ejecutar. Cubrir funciones no demuestra ausencia de errores: el significado depende del criterio de cobertura.

C. Proyectos de un curso → Mochila

Cada proyecto es un objeto; sus horas son el peso y su beneficio educativo estimado es el valor. Si hay 80 horas, la capacidad es 80. La solución indica qué proyectos elegir. Si comparten trabajo o tienen dependencias, la suma de horas puede no representar la realidad y será necesario ampliar el modelo.

Una rutina para modelar

1. **Decidir.** ¿Qué opciones se eligen, asignan u ordenan?
2. **Restringir.** ¿Qué condiciones son obligatorias?
3. **Medir.** ¿Qué costo se minimiza o beneficio se maximiza?
4. **Transformar.** ¿Qué problema fundacional reproduce esas condiciones?
5. **Interpretar.** ¿Cómo vuelve la solución algorítmica al contexto original?
6. **Validar.** ¿Qué supuestos podrían fallar en la aplicación real?

Resolver problemas difíciles en la práctica

NP-hard no significa que todas las instancias sean intratables. La complejidad describe el comportamiento general, especialmente el peor caso. El tamaño de las entradas y su estructura pueden permitir soluciones útiles.

Estrategia	Qué ofrece	Qué revisar
Algoritmos exactos	Optimalidad o prueba de inexistencia.	El tiempo puede crecer mucho.
Aproximación	Una garantía matemática de calidad.	Los supuestos bajo los que vale.
Heurísticas	Soluciones prácticas, a menudo rápidas.	No garantizan cercanía al óptimo en general.
Casos especiales	Algoritmos eficientes para cierta estructura.	Que la aplicación cumpla esa estructura.
Métodos parametrizados	Aprovechan un parámetro pequeño.	Qué parámetro controla el costo.

Distinciones que cambian la respuesta

La cobertura de vértices es polinómica en grafos bipartitos. Subset Sum admite programación dinámica seudopolinómica. Mochila 0/1 tiene esquemas de aproximación con garantías fuertes. TSP métrico admite garantías que no se trasladan automáticamente a costos arbitrarios.

Una reducción de decisión puede aumentar el tamaño de la instancia y no conservar razones de aproximación. Conviene elegir tanto el modelo como el algoritmo según el tiempo disponible y la calidad necesaria.

La pregunta útil

¿Qué tamaño tienen mis instancias, qué estructura poseen y qué calidad de solución necesito dentro del tiempo disponible?

Actividad integradora

Una ciudad tiene presupuesto para instalar tres estaciones de servicio comunitario. Cada ubicación candidata atiende determinados barrios. Algunas ubicaciones no pueden seleccionarse juntas. Un equipo de mantenimiento debe visitar las estaciones elegidas y regresar a su base.

Preguntas para trabajar en equipos

1. Cobertura. ¿Qué problema permite elegir tres ubicaciones para atender a la mayor cantidad de barrios?

2. Conflictos. ¿Cómo representarían dos ubicaciones incompatibles?

3. Ruta. ¿Qué problema modela visitar las estaciones elegidas y regresar a la base?

4. Integración. ¿Por qué resolver cada parte por separado podría producir una mala decisión global?

5. Datos. ¿Qué información real necesitan para validar el modelo?

6. Presentación. Entreguen entradas, salida esperada, objetivo, restricciones y una instancia pequeña resuelta a mano.

Orientación para el docente

Cobertura máxima representa el alcance con presupuesto. Un grafo de conflictos y condiciones de conjunto independiente representan incompatibilidades. TSP o una variante de ruteo describe el recorrido. Un modelo integrado, por ejemplo MIP, puede coordinar las tres decisiones. Para un recorrido vial habrá que obtener costos de desplazamiento apropiados antes de formular el TSP.

Criterios de revisión

Evaluar que el modelo represente las condiciones relevantes, que la dirección de la transformación sea correcta y que el equipo explique cómo interpretar la solución. Pedir que identifique al menos una limitación de su propuesta.

Referencias y precisiones

Fuentes para profundizar

Tim Roughgarden. Algorithms Illuminated, Part 4: Algorithms for NP-Hard Problems. El esquema «Eighteen Reductions» de la imagen proporcionada es la base de la selección de problemas y flechas.

Michael R. Garey y David S. Johnson. Computers and Intractability: A Guide to the Theory of NP-Completeness. Referencia clásica sobre problemas y reducciones.

Jon Kleinberg y Éva Tardos. Algorithm Design. Diseño de algoritmos, modelado y NP-completitud.

David Kempe, Jon Kleinberg y Éva Tardos. Maximizing the Spread of Influence through a Social Network (2003). Modelos y algoritmos de maximización de influencia.

Sobre el punto de partida

El teorema de Cook–Levin establece la NP-completitud de SAT. Para obtener la NP-completitud de 3-SAT se utiliza además una reducción de SAT a 3-SAT. La anotación de la imagen abrevia esta relación.

Convenciones empleadas

Min-Cost k-Path cuenta aristas, exige camino simple y tiene extremos libres. Cycle-Free Shortest Path permite pesos negativos y prohíbe repetir vértices. Makespan usa máquinas idénticas y trabajos independientes sin interrupciones. TSP se presenta en versión simétrica. Las variantes de extremos y costos se explicitan cuando afectan una reducción.

Alcance del material

Las aplicaciones son ejemplos de modelado y no afirmaciones de equivalencia para todo sistema real. Las reducciones se presentan a nivel didáctico: una prueba formal debe describir la construcción, acotar su tamaño y demostrar ambas direcciones de la equivalencia. Para reconstruir soluciones también debe explicarse su traducción.